Self-Training with Structured Data for Efficient Insect Pest Detection

Bachelor's Thesis

Christoph Wald
Matrikelnummer 17515905
christoph.wald@stud.uni-goettingen.de

Supervisors:
Prof. Alexander Ecker (Georg-August-Universität Göttingen)
Dr. Elias Böckmann (Julius-Kühn-Institut Braunschweig)

November 14, 2025

Contents

1 Introduction

Insect pests are responsible for significant production losses in agriculture [1]. However, pesticides threaten the environment by reducing biodiversity [2], particularly among insects, including pollinators that support agricultural production [3]. Overuse of pesticides causes resistance in pests. Additionally, global warming leads to increasing pest populations, which aggravates the problem further [1]. To address these challenges, the concept of integrated pest management (IPM) was developed. As defined by the Food and Agriculture Organization of the United Nations (FAO), it sets the goal of minimizing the use of pesticides by applying alternative management strategies wherever possible [4,5]. IPM is also the legal basis for the use of pesticides in the EU [6], which, among other things, prescribes the monitoring of insect pests when using pesticides. Precise monitoring facilitates the early and targeted use of pesticides for only small infested patches, the use of less harmful pesticides, or alternative ways of control like beneficial insects. Monitoring is usually based on setting up a grid of sticky traps, which are regularly checked for trapped insects. Because the manual counting requires expertise and takes a lot of time, it is not feasible for larger applications [7]. Hence, computer vision-based systems for automated monitoring of insect pests are considered to be part of the solution. Ongoing efforts to automatize insect pest monitoring have more recently adopted the methods of deep learning (see the surveys Teixeira et al. 2023 [8] and Mittal et al. 2024 [9]). My thesis tests a method to facilitate the implementation of deep learning-based, automated insect pest monitoring.

The dominant approach in pest monitoring in general involves the supervised training of YOLO models. YOLO ("You Only Look Once") is a deep learning architecture that unifies the tasks of object localization and classification into a single step. It was introduced by Redmon et al. in 2016 [10] and has been improved on since then in several iterations by different contributors. For greenhouses, specifically designed neural network models have been employed successfully in 2021 [11,12], but the recent studies from 2023 onward use YOLO models on individual datasets [13,14,15,16]. These six papers all report high performance, with mean average precision (mAP) or F1 scores ranging from 0.89 to 0.94.

Main challenges in the implementation are the small size of pests—which, even with high-resolution cameras, occupy only a very tiny fraction of an image—and the high degree of morphological variation among individuals. In less controlled environments additional complications arise from high morphological similarity between different species. These challenges create a demand for large annotated datasets; however, the collection and labeling of biological data require considerable effort and expertise. As a result, datasets are often small and exhibit other limitations, such as class imbalance. For an overview of

these specific challenges, see Teixeira et al. (2023) [8]. As a remedy, the authors recommend semi-supervised learning (SSL), a family of methods for training with only partially data. Despite this, no survey reports applications of SSL in the field of pest monitoring in general and only one study in the context of the greenhouse makes use of SSL methods: Rustia et al. [17] proposed a self-training pipeline as a supplement to a fully supervised model as early as 2021. While the results were promising, the implementation was highly intricate, involving multi-stage setups with hand-tailored solutions. This complexity may explain why the approach has not been further explored. Outside of the greenhouse, in open-field settings, some efforts were made from 2024 onward, either focusing on a single insect species with more diverse backgrounds or on a set of larger insects [18,19,20,21,22]. Another notable paper in this context is Dang et al. (2024) [23], who employ a field dataset of ten larger insect species and perform segmentation without manual annotation by leveraging a grounding language–image model. While extending this approach with classification and adapting it to the smaller insect pests typically found in greenhouses might be feasible, grounding models will not be considered in this study due to their limited speed and efficiency compared to YOLO-based methods.

Table 1 below shows an overview over the relevant literature. Green indicates a correspondence with the demands of my project, red the opposite, and orange overlappings. Because of the differing datasets and metrics, the results are not directly comparable. Also not seen in the table are the pest classes included in the datasets: Only Rustia et. al [11] covered the same pest classes featured in this project. As of now no YOLO model has been trained to identify common greenhouse pests without relying on supervised training. Although working solutions for automatic insect pest monitoring in the greenhouse have been successfully tested, practical deployment stagnates and is hindered by the need for large, manually annotated datasets for new target domains. To my knowledge, there is no adequate method to reduce the amount of labels needed. The goal of the thesis is therefore an improvement in implementation efficiency by adapting the labeling and training method.

Authors (Year)	Greenhouse	YOLO	Training
Li et al. 2021 [12]	yes	no	supervised
Rustia et al. 2021 [11]	yes	no	supervised
Rustia et al. 2021 [17]	yes	no	SSL (add. to supervised)
Costa et al. 2021 [13]	yes	yes	supervised
Zhang et al. 2023 [14]	yes	yes	supervised
Li et al. 2024 [18]	no	no	SSL (teacher-student)
Majewski et al. 2024 [19]	no	yes	SSL (pseudo-labels)
Wang et al. 2024 [15]	yes	yes	supervised
Zhou et al. 2024 [20]	no	no	SSL(teacher-student)
Gomez-Zamanillo et al. 2025 [22]	no	no	supervised, add. image processing
Shi et al. 2025 [16]	yes	yes	supervised
Zhou and Shi 2025 [21]	yes	no	SSL(teacher-student)

Table 1: Overview over relevant papers. Colors indicate overlaps to my study with green: identical, orange: overlap, and red: different; yes/no indicate if the respective paper is settled in the greenhouse context or uses a YOLO-model respectively. None of these studies uses a basic self-training approach and none tries any type of SSL with a YOLO-model in the greenhouse setting.

In this thesis I test the proposition that given a specifically structured training dataset, a model can be trained with manual labels needed only for validation and testing. This should be achieved by a combination of image processing and a semi-supervised learning method (SSL) called self-training. The key requirement is the specific structure of the dataset, which should be partitioned into subsets. All images in one subset should only contain one specific pest class as objects.

As a reference point, a supervised YOLO model is trained with manual labels, which should achieve at least 0.9 precision and 0.8 recall per class. Precision is prioritized over recall for two reasons. First, recall can be improved later through denser placement of sticky traps, whereas precision depends primarily on model quality. Second, low precision risks erroneous pesticide use, which conflicts with the principles of IPM.

The envisioned adapted automated labeling and SSL is done in two stages. First, bounding boxes for the pest classes are created with segmentation algorithms. Since the images are presorted to contain only one pest class, the detected bounding boxes can be directly associated with the corresponding label. In the second stage, the created datasets are used for self-training a YOLO model. A more detailed overview over the procedure follows in the next section.

2 Methods

2.1 Overview

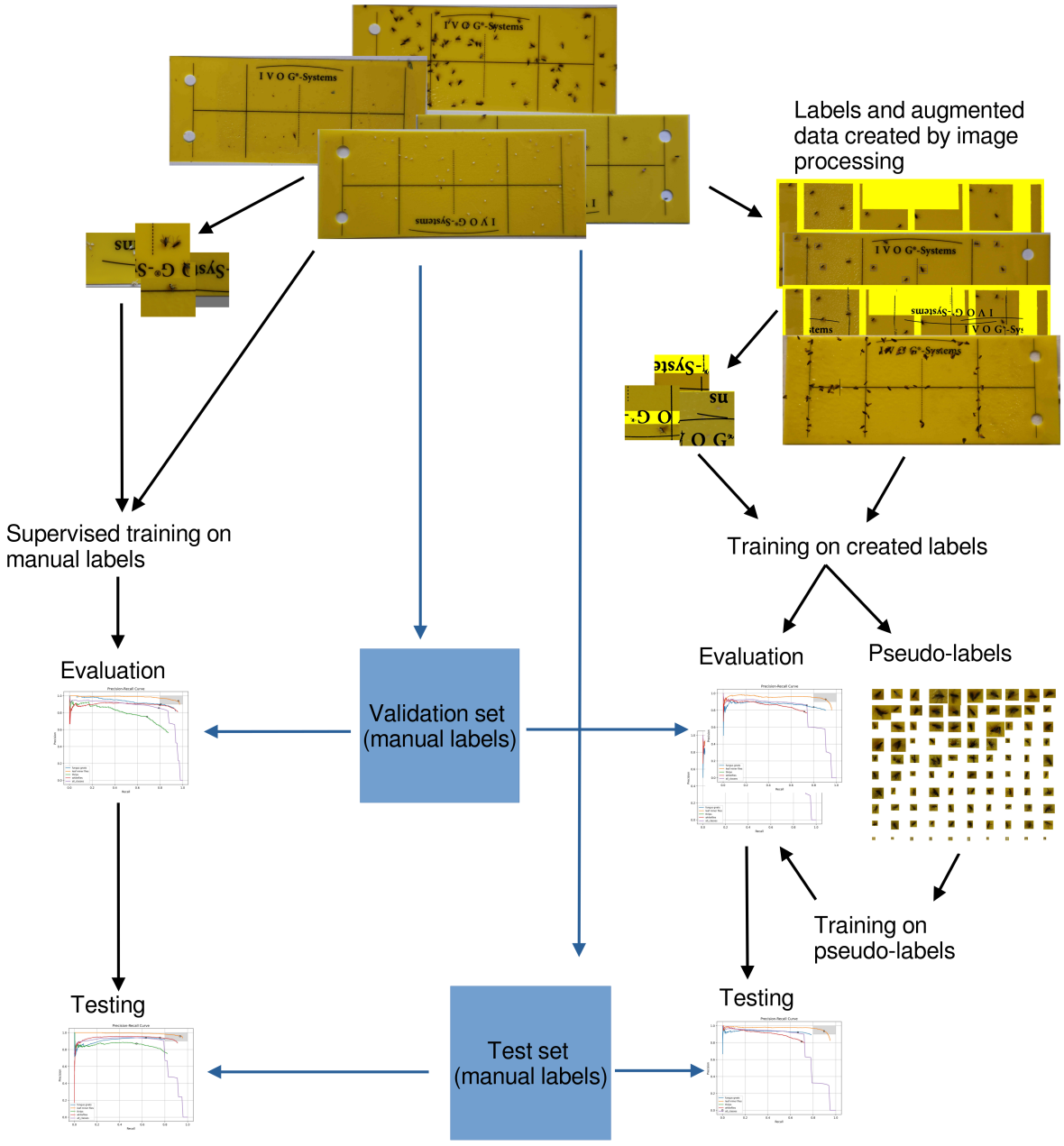


Figure 1: Overview over the experimental setup.

Main goal of the experiments is to compare a supervised model to a model trained without manual labels. After cleaning and splitting the data (see Section 2.2) a evaluation pipeline is setup to test and compare the models (see Section 2.3). Supervised training was done on tiled and full images and the best model was chosen as baseline (see Section 2.4). Image

processing was used to create labels and augmented training data (see Section 2.5) for the alternative approach. These were then used to train models, again both on tiled and full images (see Section 2.6). To improve the model further, false positive prediction were selected as pseudo-labels for retraining the model. Finally the best model was compared to the baseline.

Code and results can be accessed here: https://github.com/ChristophWald/insect_pest_detection.

2.2 Data

2.2.1 Dataset description

The data is provided by the Julius-Kühn-Institut (JKI) and was collected as part of the Smart-Checkpots project [24] where it was labeled manually and used to train a supervised model. The images are divided into four subsets each associated with only one of four insect pest classes (see Table 2) to simplify the task of manual labeling. Every image in a subset is supposed to depict several instances of only one pest class trapped on a yellow sticky trap (YST) with black background structures. Finding the optimal colors and contrast to attract specific pest classes is a topic of ongoing research (see e.g. [25]). Yellow/black was chosen for this dataset, because it is a compromise for the considered pests and also an industry standard.

Common Name	Scientific Name	EPPO Code
Fungus gnats	Bradysia impatiens, Johannsen 1912 (Diptera: Sciaroidea)	BRAIIM
Tomato leaf miner	Liriomyza bryoniae, Kaltenbach 1858 (Diptera: Agromyzidae)	LIRIBO
Western flower thrips	Frankliniella occidentalis, Pergande 1895 (Thysanoptera: Thripidae)	FRANOC
Greenhouse whitefly	Trialeurodes vaporariorum, Westwood 1856 (Hemiptera: Aleyrodidae)	TRIAVA

Table 2: Overview of insect pest classes present in the dataset.

The chosen insect pests are common in greenhouses, particularly on crops such as tomatoes and cucumbers, where they can cause severe damage, up to complete crop loss [11]. As Figure 2 shows, the size of the pest classes and thereby the number of pixels they cover differs considerable. In thrips, also male and female differ in appearance. The female in-

dividuals are larger and have good contrast against the yellow background, whereas the smaller and more transparent males are harder to detect (compare e.g. row 3, box 2 and 3).

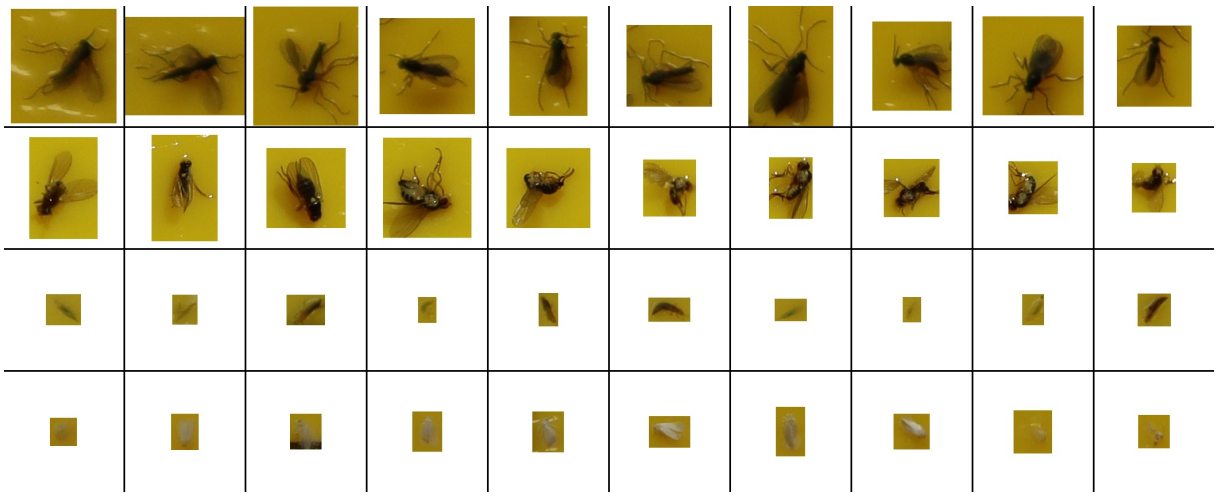


Figure 2: Cropped boxes with fungus gnats, leaf miner flies, thrips, and whiteflies (top to bottom). Original resolution to compare sizes.

The populated YSTs in this dataset were created at the JKI by breeding the insects to the adult stage in a closed environment (see Figure 3a). For the fungus gnats and the leaf miner flies the YSTs were exposed in the environment for some time until it became populated. For the whiteflies and the thrips the tapping method was used, where the YST is tapped against the plant to catch the insects. After a batch of YSTs was populated, pictures were taken immediately with a Canon EOS M50 Mark II on a tripod at a distance of approximately 13 cm (see Figure 3b) and a resolution of 6000 $\times$ 3368 pixels, resulting in files of about 5–7 MB. No artificial light was used to avoid reflections. The pictures have slightly different view angles and different lighting conditions, some still affected from reflections and some having areas with low sharpness.

(a) Breeding station

(b) Camera setup

Figure 3: Data creation, pictures with kind permission M. Pollmann, JKI

While the YSTs were captured, few insects could leave the trap and were then trapped by another trap next to it. Therefore the subsets can contain some insects belonging to other subclasses. The tapping method for the whitefly also catched some parts of insects and some foliage of the tobacco plants. Thrips were bred on bean plants which left considerable amount of foliage on the YSTs. Also, thrips in larval stages are trapped, which are hard to distinguish from male individuals. In the use case, larvae will cannot be trapped, because only adult can fly.

For each pest class a proportion of images were manually labeled by several persons. Visual inspection showed inconsistency in the correctness of labels, the completeness of labels and the size of the bounding boxes.

2.2.2 Preprocessing

To raise the quality of the dataset, images and labels were checked and improved where possible.

Image files were sorted by their creation time given by the metadata and then the first 100 images for each class were visually inspected, which contained several images with identical YSTs and different camera setting as leftovers of early experiments with the camera setup. These were removed manually. I removed further duplicates (identical images) by comparing file checksums. Images with labels that indicated no insects or unexpected insects not belonging to the class of the subset were also removed.

For the thrips, I inspected the complete subset before splitting the dataset. This led to the exclusion of 200 images with wrong labels or too much other objects. To improve the quality of the thrips labels further I gave the cut out bounding boxes to an expert from the JKI, which identified 1276 labels as incorrect. These were removed. Table 3 shows the number of images and labels after the cleaning reported. The dataset is imbalanced, both in terms of the number of labeled images per class and the mean number of instances per image.

Class	Images	Labeled	Fraction labeled	Instances labeled	Mean instances per image
Fungus gnats	1328	1018	76.66 %	33373	32.78
Leaf miner flies	1422	659	46.34 %	6518	9.89
Thrips	1134	392	34.57 %	5453	13.91
Whiteflies	1105	433	39.19 %	25735	59.43
Total	**4989**	**2502**	**50.15 %**	**71527**	**28.59**

Table 3: Annotation statistics after cleaning and preprocessing.

Because of the reported inconsistencies in the labeling, the test set comprising 289 images was checked and corrected by an employee of the JKI, which took about x hours. Unfortunately time constraints prevented a relabeling of the training set, such that supervised training and all validations had to be carried out on a label set of lower quality. The results of the supervised training therefore provides only a lower bound for the expected quality of a model.

Class	Kept labels	Added labels	Deleted labels
Fungus gnats	1487	549	92
Leaf miner flies	1241	59	18
Thrips	1113	425	65
Whiteflies	2520	194	102
Total	**6361**	**1227**	**277**

Table 4: Revised label counts for the test set compared to the initial annotations. Leaf miner flies show the smallest changes, while thrips and fungus gnats had a large proportion of previously unlabeled objects. For comparing the evaluation procedure detailed below were used.

The summed difficulties with the thrips images and labels led to an exclusion of the class for all experiments but the initial supervised training, which showed that no satisfying results are to be expected in the proposed self-training. There are three major reasons for this decision: low sample size, presence of non-target objects, and low label quality. Because of size and morphological differences between adult males and females the thrips is the hardest pest class to detect and has to be represented with an according number of images and instances in the dataset. Unfortunately, after cleaning the thrips subset has the lowest number of images and labels in the dataset. Furthermore the thrips are

easiest to be mixed up with other objects like foliage and therefore require clean images. But the thrips images are the only pest class that has lots of non-target objects, including thrips larvae on their images. The 1276 false labels detected in the inspection of the cut out boxes and the 65 additionally identified false labels in following revision of the test set clearly show that the automated labeling cannot work under the assumption that all objects on an image are instances of the respective pest classes. Finally, the inconsistent labeling quality prohibits proper testing.

Some labels were found to contain negative coordinates. After visual inspection of samples, these values were interpreted as their absolute counterparts, since this matched the actual object positions. The underlying cause of the negative coordinates could not be identified but seems to be connected to the export of labels from Label Studio.

2.2.3 Splitting

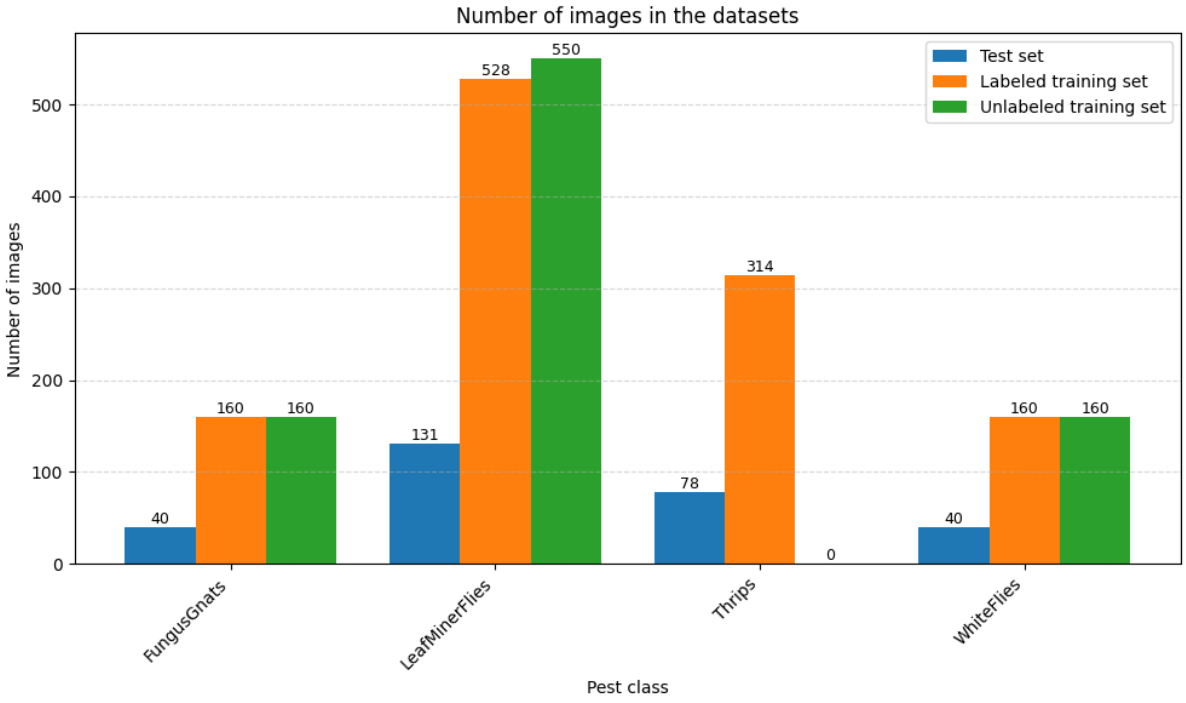


Figure 4: Number of images in the randomly created subsets. Because the absolute number of species per images is different for each class, the number of included images varies.

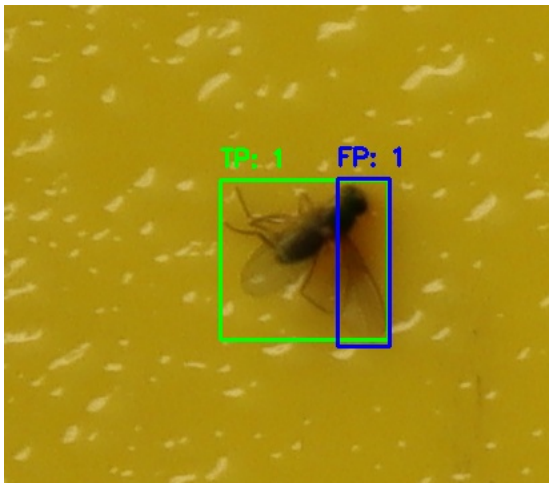
Because the number of insects per images is different for the pest classes (see Table 3) sets were balanced where possible. From the labeled images, 200 images for the fungus gnats and the white flies were drawn randomly. For the other classes all labeled images were used. The labeled set was split into 20% test set and 80% training set, and the

training set was further divided into a training and validation set with the same ratio. From the remaining images an unlabeled set with comparable size was drawn to replicate the results of the image-processing and self-training.

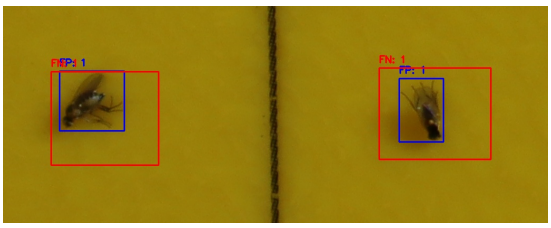
2.3 Evaluation

The evaluation pipeline consists of collecting the predictions of one model, comparing them to the ground truth, finding the best operating points per class and finally comparing the models to each other. Results of parameter tuning (IoU thresholds, stride values, and filter thresholds) are summarized in Table 10. They were tested on two early models trained on tiles with confidence threshold set to 0.5 for all classes, non-maximum supression threshold for filtering before the ground truth to 0.4 and IoU threshold for comparision to ground truth to 0.5 in all tests.

The used YOLO-model only allows prediction on full images and also only allows one confidence threshold for all classes. I used a wrapper script to enable prediction on tiled images and prediction with class specific thresholds. If the prediction is on tiles, a sliding window with a set stride collects the predictions for one image. These are then filtered by a non-maximum suppression (NMS) filter and another filter to exclude contained boxes. NMS relies on the intersection of union (IoU) metric, which is the ratio of the area of two boxes intersected and the area of the union. When two boxes have an IoU above a threshold, only the box with the higher confidence is kept. However, if a small box is completely or partially inside a larger box, the IoU will be low, because the intersection is only as small as the small box and the union is as big as the large box. As visual inspection shows, the small objects in the dataset make it necessary to include an extra filtering for this case. See Figure 5 a for false positives that occur without the extra filter. IoU for NMS was set to 0.4 and the threshold to filter out the other case was set to 0.5. For the stride the values 420 and 440 were tested.



(a) Overlapping bounding boxes not detected by NMS (leaf miner fly), which creates a false positive box.



(b) Bounding boxes counted as FP because of low IoU with the larger ground truth box (leaf miner flies).

Figure 5: Detection errors caused by low Intersection over Union (IoU) with small boxes. Green boxes are true positives (TP), blue boxes false positives (FP), red boxes false negatives (FN).

The filtered predictions are then sorted by confidence and are attributed to possible ground truth boxes with greedy matching. Attribution is again defined by IoU and another containment filter, that is implemented, because some manual labels are large, such that much smaller but correct predictions are contained in the ground truth box and produce a low IoU value. See Figure 5 b for false positives that occur without the extra comparision. This evaluation logic was also used for comparing the old and new labels for the test set and for evaluating the final results of the image processing. For IoU a threshold of 0.5 was set, for the extra filter thresholds between 0.5 and 0.9 were tested.

For defining the quality of a model, per class precision-recall curves are built and confidence thresholds that are closest to the target zone of precision greater or equal than 0.9 and recall greater or equal 0.8 are taken as operating points for the model. If the curve reached the target zone, the point with the best F1 value inside the zone was chosen. Different models are then compared by the number of operating points inside the target zone and the distance of the remaining operating points to the target zone.

2.4 Supervised training

I used a YOLOv8 model. I trained on the full images resized to 1024 internally by YOLO and by two different sets of 640x640 segments built with stride 440. The sets differ by the inclusion strategy: The first set simply includes all resulting tiles. In the second

tiling approach, for each object only one tile was allowed to avoid redundancy in the training set and tiles with partial objects were dropped. Additionally for the first set the proportion of empty background images included and the exclusion of the thrips were tested. Augmentation was applied only carefully.

As a reference point for the automated labeling process, a YOLOv8 model is trained using the manual labels provided. The aim is to demonstrate that the data is sufficient to reach the target metrics of a minimum per-class precision of 0.9 and recall of 0.8. Since supervised training is not the main focus of this project, experiments are not designed to fully optimize results. A small YOLOv8s model pretrained on COCO is chosen because initial tests showed that larger models or later versions did not improve performance. With 4-5 GB of weights, the small model also meets the requirement of single-GPU training and the more modular code of version 8 is easier to adapt for self-training. No task-specific pretrained models are available to my knowledge.

The images are cropped to the size of the YST with color threshold based segmentation (compare YSTs in Figure **??** and Figure **??**). This decreases the size of the dataset by 17,1 GB but also introduces different image sizes. Labels are therefore recalculated after cropping. Since YOLO labels are relative to image width and height, the new image dimensions must be used when transforming labels back into absolute coordinates.

YOLO expects images of size 640x640 and resizes images with larger resolution. Because the insect pest classes are of small size, this resizing might cause difficulties. Therefore images are tiled. After padding each image such that its dimensions are divisible by 640, the images are tiled into 640×640 patches with a stride of 440. The overlap of 200 pixels corresponds to the size of the largest pest class, the fungus gnats, such that an instance is at least contained in one tile. The labels are recalculated according to the tiles and labels and are only kept if they are contained to at least 80% inside the respective tile. The resulting tiles contain a large number of empty background tiles, which account for over 68% of the tiles in the labeled training set. For the training on the full images, the rescaling parameter in YOLO was set to 1280.

Starting from the default settings, short training runs with selected parameter changes were conducted to identify optimization potential, where augmentation settings and the ratio of background to foreground images are identified as the main factors influencing performance. Changing the default optimizer (stochastic gradient descent), learning rate schedule, and loss weights did not yield improvements. Because the dataset already contains considerable variance (see Section 2.2.1), augmentations are applied only subtly. The scaling factor is reduced to avoid unrealistic size differences between objects, and the probability of mosaic augmentation is lowered to prevent the model from learning

lighting differences between images. A small amount of mixup is introduced with a 5% probability. All other parameters are kept at the default values. (I could put the full list of arguments in the appendix).

All models were trained up to 200 epochs, then the best epoch (in terms of mAP@90:95) was taken.

Training run	Dataset	Background ratio	epochs trained	Best epoch
train1	tile v1	68%	200	144
train2	tile v1	30%	200	127
train3	tile v1 without thrips	200	47%	136
train4	full images	0 %	100	94

Table 5: Selected training runs for supervised training

For the training runs the proportion of background tiles (tiles without objects) is varied (see Table 5). The initial training contains 68% background tiles, which is reduced to 10% and 30% in a number of training runs. To test the effect of the thrips images on the training of the other classes, the set with only 30% background images is stripped from all images from the thrips subset which changed the background to foreground ratio to 47%.

2.5 Image processing

The aim of the image processing is to detect a proportion of objects and create labels including bounding box coordinates, which will serve as the starting point for semi-supervised training of a YOLO model. The simple main idea of this step is to separate background and foreground with a color threshold and then keep all coherent foreground contours as objects. A rectangle around the contour serves as bounding box and the class id can simply be derived from the subset of the image, as all insects on an image are of the same pest class. For images without background structures the process reduces to the short and fast procedure reported in section 2.5.2. However, when images contain background structures—as in the dataset used in this study—more complex preprocessing is required, as detailed in Section 2.5.1. The presented pipeline was developed with the labeled training set. In between evaluation is done on the ground truth given by the manual labels. Raising the absolute number of true positives and minimizing the number of false positives are the main metrics. Precision is also tracked but is considered secondary, as it might come at the cost of too less true positives.

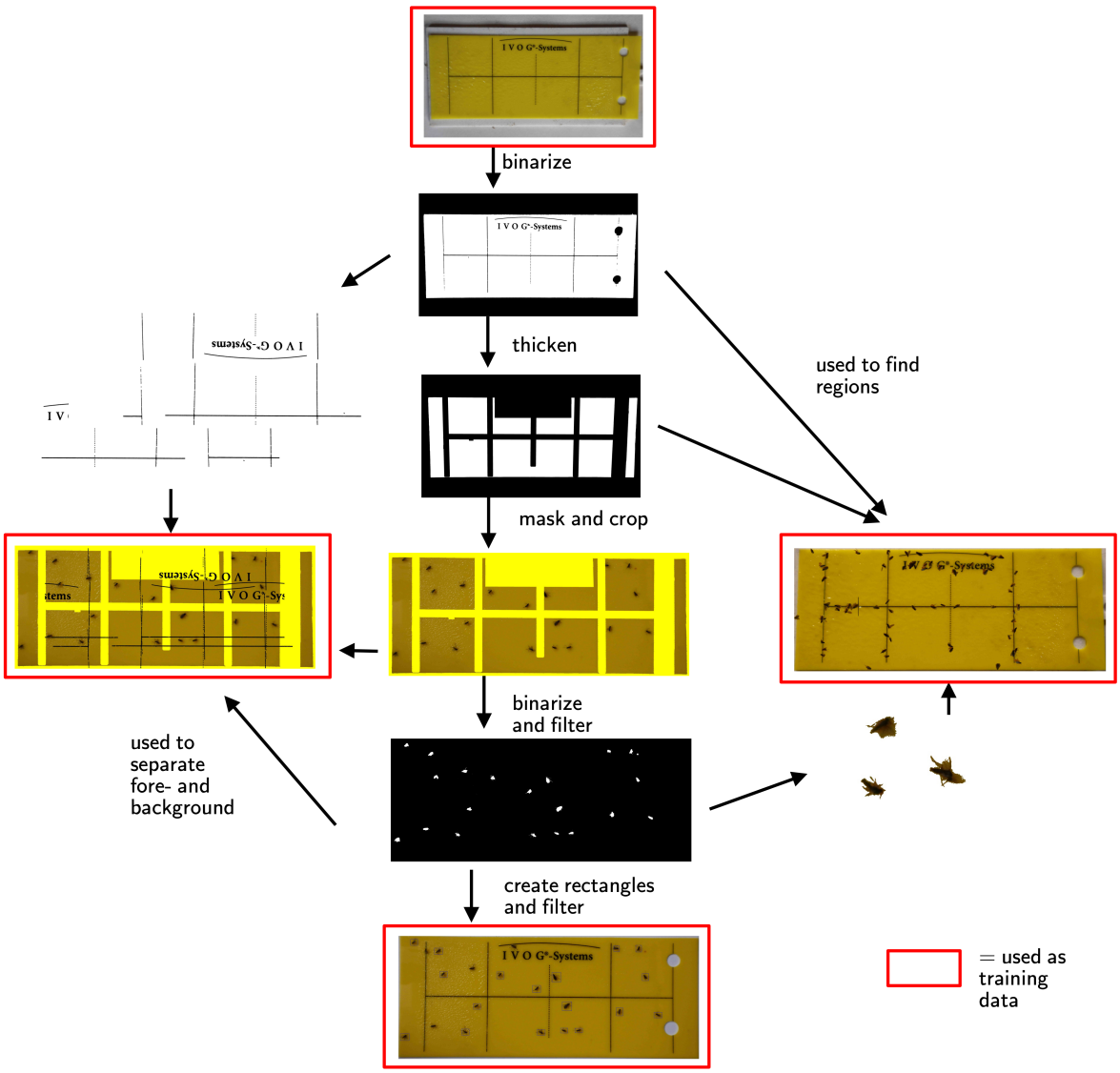


Figure 6: Overview over creating training data with image processing.

Possible example images not included: LIRIBO_0614 for effect of shifting the amsk, TRIAVA_0037 for effect of fatter maks, TRIAVA 0320 for missing unsaturated whiteflies, TRIAVA 506 for the detrimental effects of other filtering, all files in image Processing evaluation - mask tests

2.5.1 Preprocessing for segmentation

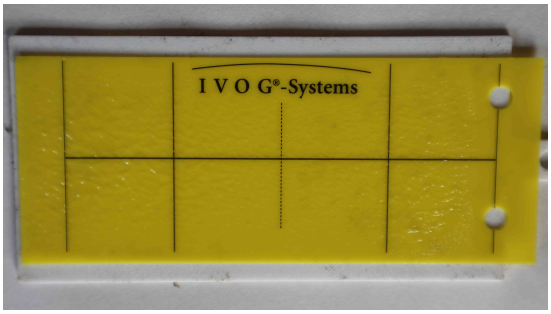
I created some masks (Figure 7) and devised a processing pipeline. The result is a binarized image which divides insects as foreground and the black structures as background .

To filter out background structures, a mask covering these regions is applied to each

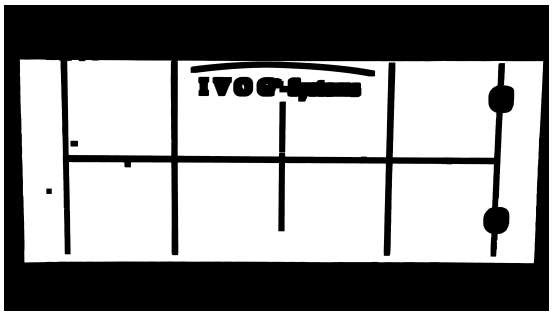
image. The pixels within the mask are changed to a yellow hue, allowing them to be identified as background during segmentation. Direct detection of the lines was tried, but led to no useable results, as the detected line segments were often very short and could not be separated from the line segments of the objects, that should be kept. Because the images in the dataset were captured from slightly different angles, the mask must be aligned individually for each image. All image processing was performed using the cv2 Python library. Binarization, which is used at several steps, is achieved through color thresholding in the HSV color space, as the hue component provides robustness against variations in lighting. The thresholds used were: Hue 20–30, Saturation 100–255, and Value 100–255.

The processing pipeline begins with a subset of uncropped images, as determined by the splitting described in Section 2.2.3. Since the masking procedure assumes the header is located at the top of the YST, images were rotated when necessary. The required rotation was detected by dividing the YST area into horizontal halves and comparing the summed black pixel area in each half, with black pixels defined as having a value below 50. This method becomes unreliable when the YST is crowded with a large number of black insects, so a visual inspection was performed to ensure accuracy. Correspondingly, the labels used for manual evaluation were rotated to match the corrected image orientation.

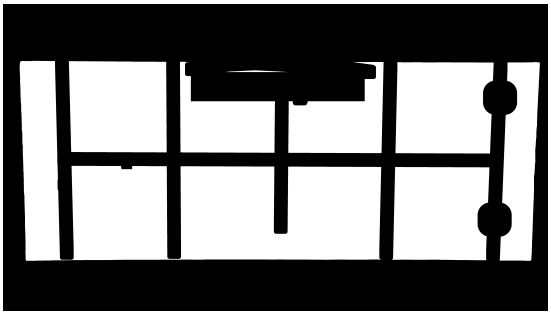
Creating the mask required balancing line thickness to fully cover background structures despite minor misalignments, while avoiding excessive coverage of the objects. Binarized and denoised images of empty YSTs, captured under the same conditions as the dataset images, were used for mask generation. The masks were created by combining several empty YST images with projective transformations and thickening the black structures woth dilation (kernel size 25) (Figure 7b). From this mask, two additional variations were generated by applying further dilation (kernel size 50) (Figure 7c) and by manually thickening the right border lines to cover the holes completely and drawing a rectangle over the header (Figure 7d). For subsequent alignment, the corners of the YST contour and two points of the horizontal midline calculated from the original masks were saved.



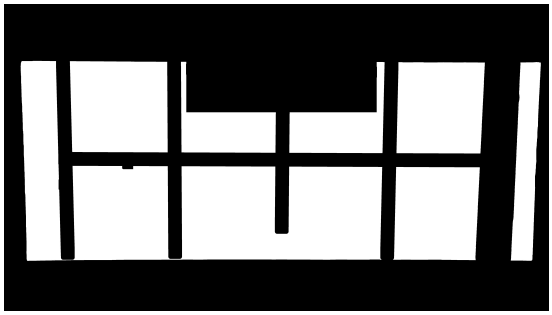
(a) Image used for the creation of masks.



(b) Mask created by overlaying other images to the initial one and then thickening the lines.



(c) Mask from b) with thicker lines.



(d) Mask from e) with a rectangle as header.

Figure 7: Masks tested for masking out the background structures.

With the masks prepared, preprocessed and masked versions of the image set were created. The pipeline of this process is divided into the following steps.

- Find contours on the binarized image. Take the largest, which is the YST. (algorithm used: cv2.findContours)
- Assuming a rectangle shape find the corners or exclude the image from further processing, algorithms used: cv2.arcLenght, cv2.approxPolyDP)
- Using the corners of the mask and the image, do a projective transformation of the mask and of the midline of the mask. , algorithms used: cv2.findHomography, cv2.warpPerspective)
- Using the middlepoints of middle horizontal lines of the mask and the image, do another affine transformation of the mask, if the difference is lower than 500 pixels. If the result is larger, a wrong horizontal line was picked and the transformation is skipped. , algorithms used: cv2.HoughLines, cv2.perspectiveTransform, cv2.warpAffine)
- Color the pixels given by the aligned mask in the image yellow (background) as seen in image .

- Crop the image to the YST (and recalculate the manual labels for evaluation).

2.5.2 Segmentation

The foreground objects need to be filtered further. For testing the masks this was done by simply thresholding the size of the foreground contours. For final use a more refined set of thresholds is derived from the distribution of of the sizes and shapes of the rectangles created around the contours. (see Table 6).

More intricate binarization for TRIAVA was tried, but was not successful. Value thresholding for TRIAVA was tried but was not successful.

The segmentation is done on the binarized, preprocessed images. The results of the segmentation are filtered by several thresholds. A first set of threshold is used to evaluate the effects of the different masks as seen in Figure 7. A second, more refined set of thresholds is derived from inspecting the data given by the best evaluated mask (Figure 7d. Evaluating was done on the manual labels of the labeled training set. Comparing to GT was reduced to IoP, because it gives nearly identical results for smaller predictions inside ground truth and is more strict for overlapping ones.)

The first set entails guessed lower and upper threshold for the area contours detected. For fungus gnats and leaf miner flies only contours with an area in the range between 1000 and 10000 pixels are kept, for the whiteflies the range is between 100 and 1000 pixels. The rectangles around the contours kept are then scale with factor 1.5, because the contours lost small features like legs, that would cause the rectangles to be too small (see Figure **??**). A second threshold then filters the rectangles by the ratio of their height and width. Assuming that more oblong rectangles are likely to be non-insect objects (e.g. left over background elements, foliage, parts of insects), the maximum ratio was set to 2. This excludes rectangles with one side being more than twice as long as the other side.

For each mask, segmentation is done for all preprocessed images and the results were evaluated by comparing to the manual ground truth with the Intersection over Prediction metric with threshold 0.5. The results from the best performing mask Figure 7d are used to inspect the distribution of box sizes, box ratios and, only for the whiteflies, the value in the HSV-space for all rectangles. Additional filter thresholds were then tested on the calculated rectangles that are given in Table 6. Goals were to reduce the number of small insect parts and occasional non-insect objects, as well as the number of large boxes with more than one insect occluding each other. Only for the whiteflies a considerable number of overlapping boxes were created, presumable because individual objects often led to two separate contours. Therefore this case was checked and only one rectangle kept.

Species	Threshold	Filter set 1	Filter set 2
Fungus gnats	Min contour	1000	*same*
	Max contour	10000	*same*
	Max box size	-	95th percentile
	Max box ratio	2	95th percentile
Leaf miner flies	Min contour	1000	*same*
	Max contour	10000	*same*
	Max box size	-	95th percentile
	Max box ratio	2	95th percentile
Whiteflies	Min contour	100	*same*
	Max contour	1000	2000
	Max box size	-	95th percentile
	Max box ratio	2	95th percentile

Table 6: Thresholds of the two filter sets for filtering the results of the segmentation.

2.5.3 Augmenting the data

Masking the background structures including the insects on it will create biased label set. A model trained on this dataset will inherit this bias and learn to to ignore insects on background structures. To prevent that, two types of augmentation sets with insects and lines overlayed are created. The first set is based on the masked images with the background structures covered. On these images background structures are distributed randomly. These cut through the labeled insects and provide examples of insects and background combined. The second set is based on images of empty sticky traps. Cut out insects are distributed on and near the the background structures.(see Figure 6 for examples).

Steps for creating augmentation set 1:

- Divide masked image into background and foreground by color based threshold segmentation.
- Get the background structures segmentation of an empty sticky trap, also by color based threshold segmentation.
- Remove unwanted parts (black borders, holes) and make a random horizontal and vertical shift such that the structures are placed on a random location on the image. The shift is rotational, such that parts leaving the image enter again at the other side.

- Repeat with a second empty stick trap rotated.
- Place the two prepared background structures on the background of the masked image and finally add the foreground with the insects at the top. This secures that insects are on top and not covered by the structures.
- Do this for the complete training set.

Steps for creating augmentation set 2:

- Take images of empty yellow sticky traps and shear them to have more variations.
- Take the bounding boxes detected by image processing and make a more refined segmentation to cut out only the insects. For whiteflies I chose fixed thresholds, then separated white pixels inside the contour and from those outside and restored them to the original color, for fungus gnats I tried compared fixed thresholds with value threshold by Otsu's method (marked as v1/v2 in the test). I also filtered the remaining contour to keep only the largest and remove artifacts.
- For the whiteflies I only introduced value variations. The ranges were taken from comparing the results of image processing with the manual labels. A significant difference in value distribution of true positives and false negatives was found. Is this worth a figure?
- On the YST-image, find the background locations by masking, choose some random insects (per class) and place them there. I used one mask for the lines and another enhanced one for the areas around the lines and the letters to cover bother cases. I also divided the image in segments and set individual numbers to distribute, to have enough insects in all background elements.
- To add more variaton, random changes in lighting and vertical orientation were applied.
- for each pest class 24 images were created to supplement the training data.

2.6 Self-Training

I first trained on the image processed labels with augmentations, where the model learned to detect objects and classify them. I then used this model to predict labels on the original dataset without augmentations such that also insects on the background structures are now labeled. I then tried to improve the model by adding more pseudo labels.

2.6.1 Training on the labels created by image processing

Run	Flipud	Epochs	Basic Data	Empty Tiles	Augmented Set 2
train1	no	10	tiles	no	no
train2	no	10	tiles	yes	no
train3	yes	20	tiles	yes	no
train4	yes	20	tiles	yes	v1, on lines
train5	yes	20	tiles	yes	v1, on and near lines
train6	yes	20	tiles	yes	v2, on and near lines
train9	yes	20	augmented tiles 1	yes	no
train10	yes	20	augmented tiles 1	yes	v2, on and near lines
train14	yes	20	augmented 1 full images	no	full images v2, on and near lines

Table 7: Training runs for training on the image processed labels. Flipud turns the image 180° with probablity 50%. Tiles is the unaugmented dataset with all tiles containing created labels. Augmented tiles 1 is the dataset as described in... Augmented set 2 are the augmentation described in .... Rescaling for full images was set 1280 as in supervised training.

- was done on tiles (as explicated in the section on supervised training)
- Learning rate (0.00125) and Optimizer (AdamW) were set automatically by YOLO.
- only tiles including at least one labeled instance were included in the basic train set
- for validation subset the manual labels are kept
- augmentation setting of yolo were taken from above (filter set 2), but also rotation was tested
- tests on inclusion of empty tiles
- tests on replacing the original dataset with the augmented set 1
- addition of augmented set 2 tested
- should be compared to full image training
- should be compared to the results on the unlabeled set

2.6.2 Further training on predicted labels

Run	thresh fungus gnats	thresh leaf miner flies	thresh whiteflies
train11 0.25	0.25	0.25	
train12 0.8	0.8	0.7	
train13 0.7	0.7	0.6	

Table 8: Training runs for the self-training with additional pseudo-labels filtered by class-specific confidence thresholds for tiles. Flipud was always set to 0.5.

Run	thresh fungus gnats	thresh leaf miner flies	thresh whiteflies
train15 0.25	0.25	0.25	
train16 0.5	0.8	0.7	

Table 9: Training runs for the self-training with additional pseudo-labels filtered by class-specific confidence thresholds for on full images. Flipud was always set to 0.5, rescaling to 1280.

- same settings as before (learning rate, optimizer, epochs = 20, flipud 0.5)
- new labels were included as pseudo-labels filtered by a confidence threshold
- predicted class was hardcoded from the species given by the filename/datasubset on the first run, where the labels were transferred from the training on the image processed labels to the original dataset with backgrounds included
- epoch length 5 &10 was tested
- inclusion of new images with new predictions was tested (on unprocessed images)
- weighing of the losses for the pseudo-labels (with disabled mosaic/mixup augmentations) was tested

3 Results

3.1 Comparing supervised training and self-training

Overall training on full images produced better results. The best model is the supervised model on the manual labels (train6). In validation it arrived the target zone for all

but the incoherently labeled thrips. Because confidence thresholds are derived from the incompletely labeled validation set, they did not provide the best performance for the model on the test set, where it missed the target zone for all but the leaf miner flies. But as can be seen in the precision-recall curve on the test set the model increases precision overall in the test set, because the more complete labels lead to lower false positive rates.

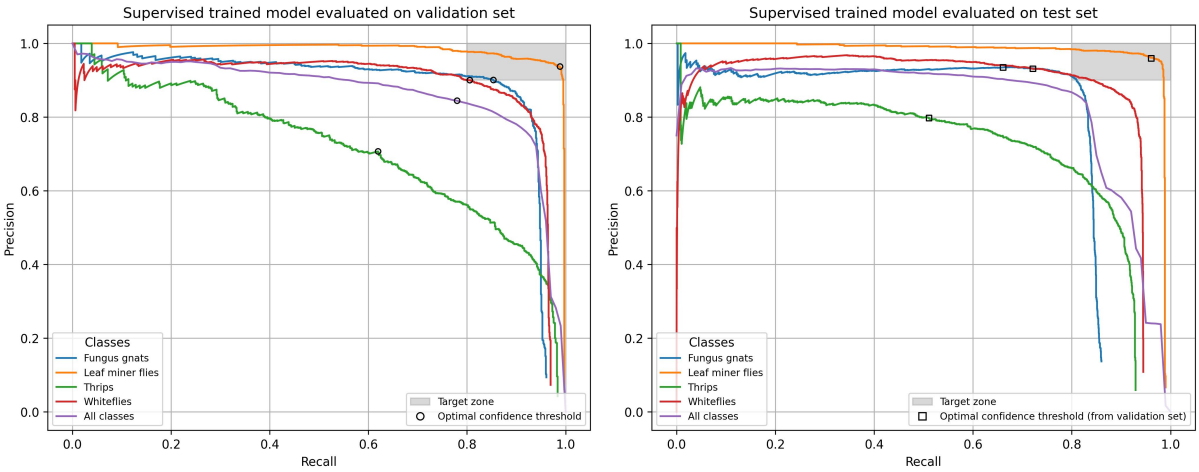


Figure 8: Precision-recall curves for the best model trained supervised on full images. The left curve is taken on the validation set with the points closest to the target zone marked The confidence thresholds associated with these points are used to predict on the test set. The right curve is taken on the test set with the arrived precision/recall values on the confidence thresholds. Training the model on the incomplete labels would still provides a sufficient model for all but the thrips, however deriving the confidence thresholds from the incomplete labels makes the model miss its best performance on the completely labeled test set.

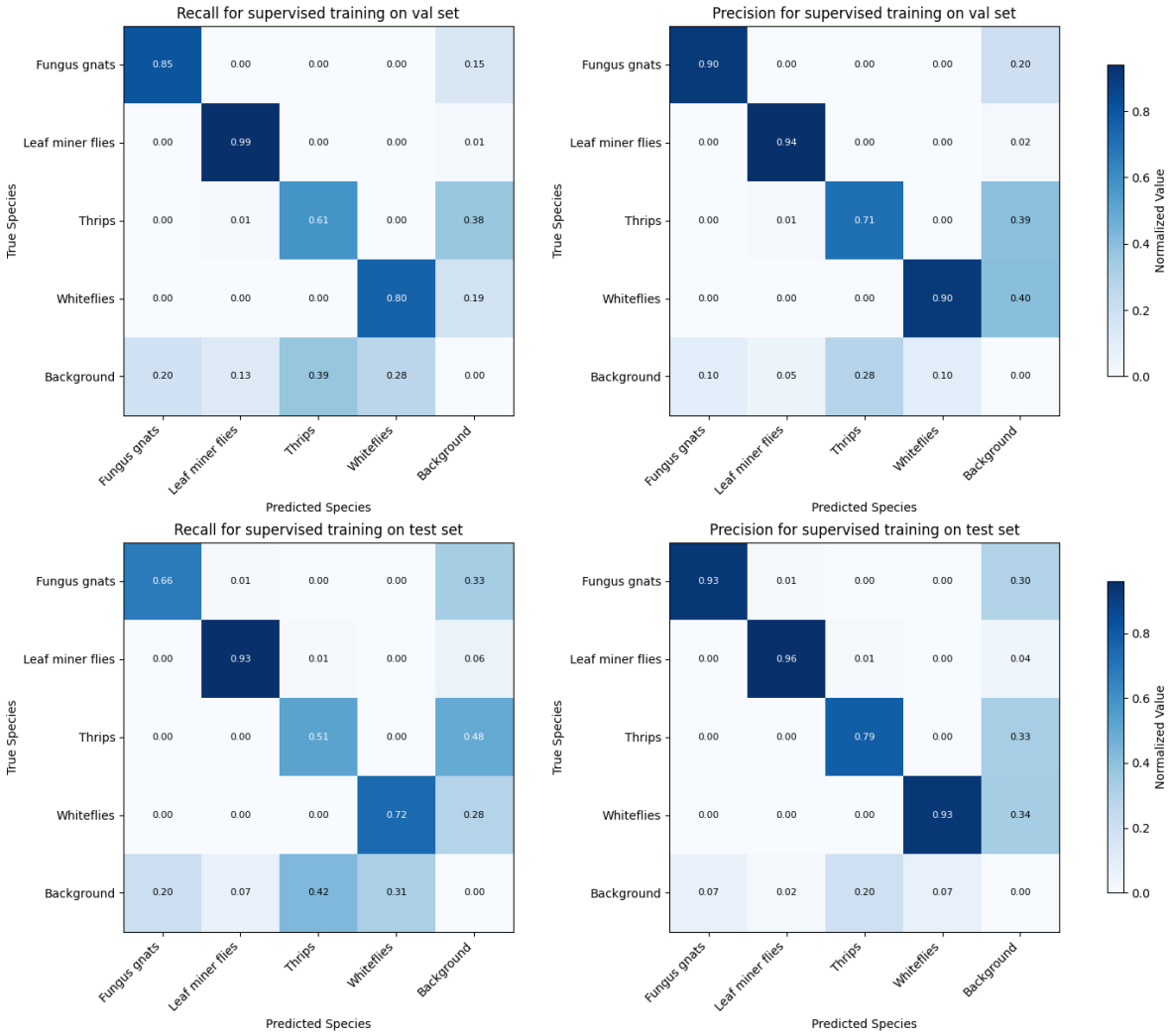


Figure 9: Confusion matrices for the supervised training on the validation set and the test set. The thresholds derived from the validation set lead to higher precision and lower recalls in the test set.

The best self-trained model (train16) arrived the target zone only for the leaf miner flies, although it improved on the image processed labels.

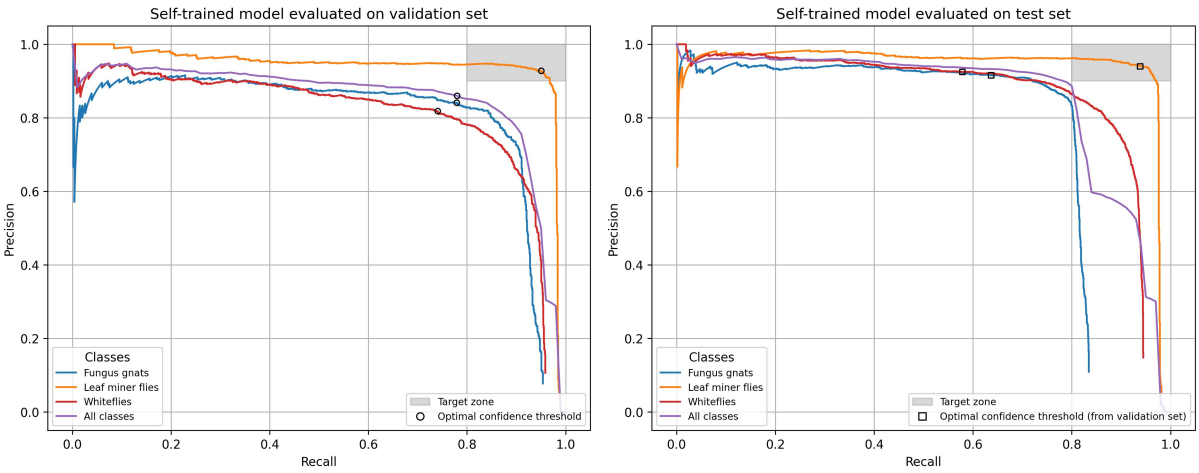


Figure 10: Precision-recall curves for the best model self-trained (on full images). The grey rectangle defines the target zone. The left curve is taken on the validation set with the operating points closest to the target zone marked as circles. The right curve is taken on the test set with the precision/recall points arrived at marked as squares. Self-training did not provide a sufficient model. (I could mark the points for the image processing to show the improvement).

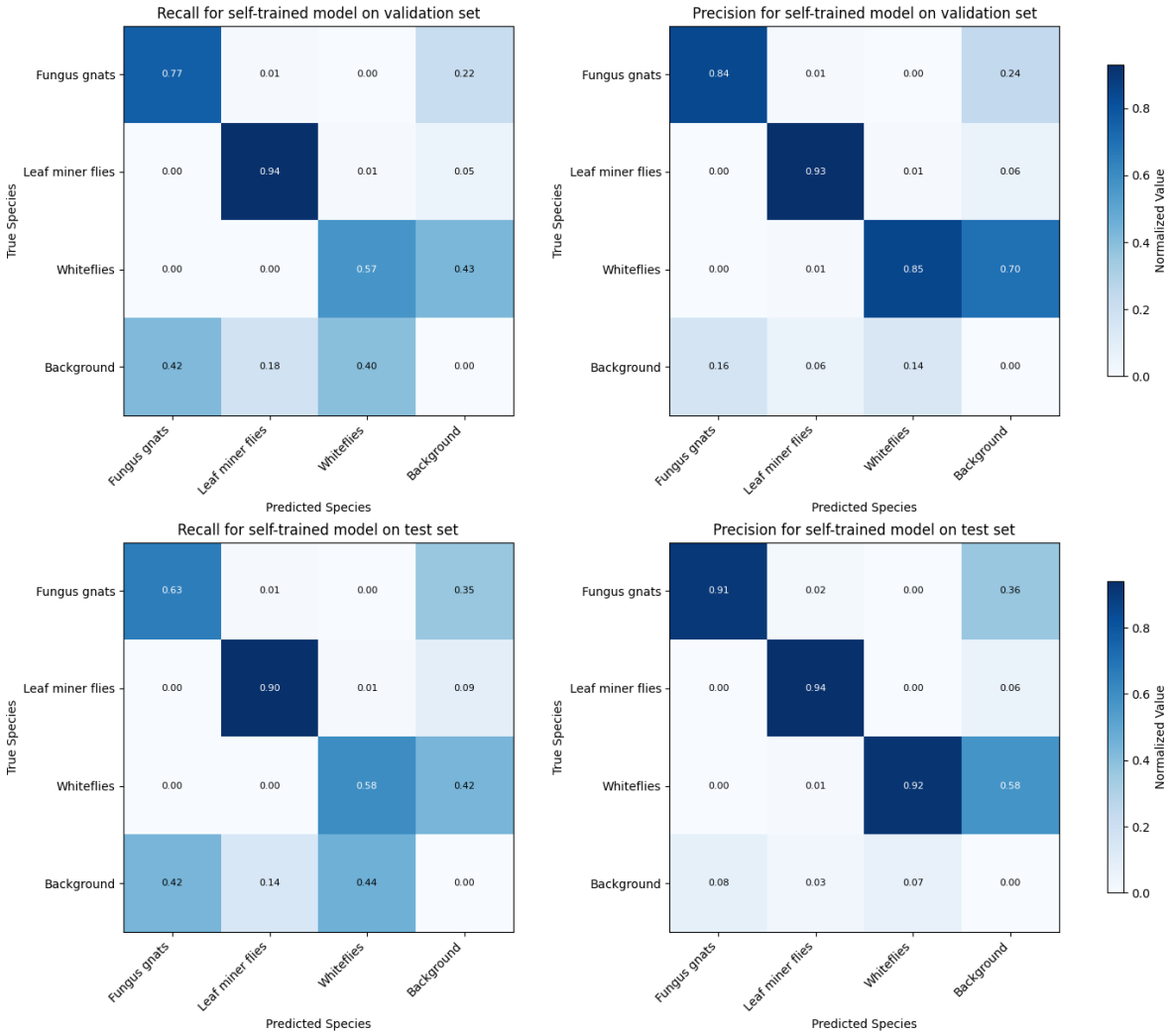


Figure 11: Confusion matrices for the self training on the validation set and the test set . Again the thresholds derived from the validation set lead to higher precision and lower recalls in the test set. Both precision and recall are improved compared to the image processed labels (cf. Fig 15), with recall including now the insects on the background structures, that were masked out before.

3.2 Evaluation settings

Final selection for all tests were stride 420 (for models trained on tiles) and threshold 0.5 for filtering predictions contained in larger predicted boxes as well as allowing contained predictions contained inside ground truth boxes. The final settings never degrades performance, but the scale of improvement differs on different models. Changing the stride to 420 always brings a boost for the number of true positives. The extra containment filter before comparing the boxes brings a slight reduction in false negatives only for train1. The extra containment section in the comparing to the ground truth with 0.9 always brings benefits in all three metrics, but setting the threshold to 0.5 only improves results

for stride 420.

Stride	Containment filter	Containment comparison	train1			train2		
			TP	FP	FN	TP	FP	FN
440	-	-	3240	637	1024	3469	664	798
440	0.5	-	3240	592	1027	3469	664	798
440	0.5	0.9	3331	501	936	3469	584	798
420	0.5	0.9	3569	540	698	3737	716	539
440	0.5	0.5	3601	508	666	3469	587	798
420	0.5	0.5	3601	508	666	3737	637	530

Table 10: Comparison of detection results under different settings of extra thresholds with given absolute numbers for true positives (TP), false positives (FP) and false negatives FN). The models are trained on tiles, each 100 epochs, train1 on the full trained label set, train2 with only 30% of the empty background tiles. The chosen parameters in the last row improve results for both models.

3.3 Supervised training

The supervised model on trained on the tiles (train4) was slightly worse then the one trained on the full images.

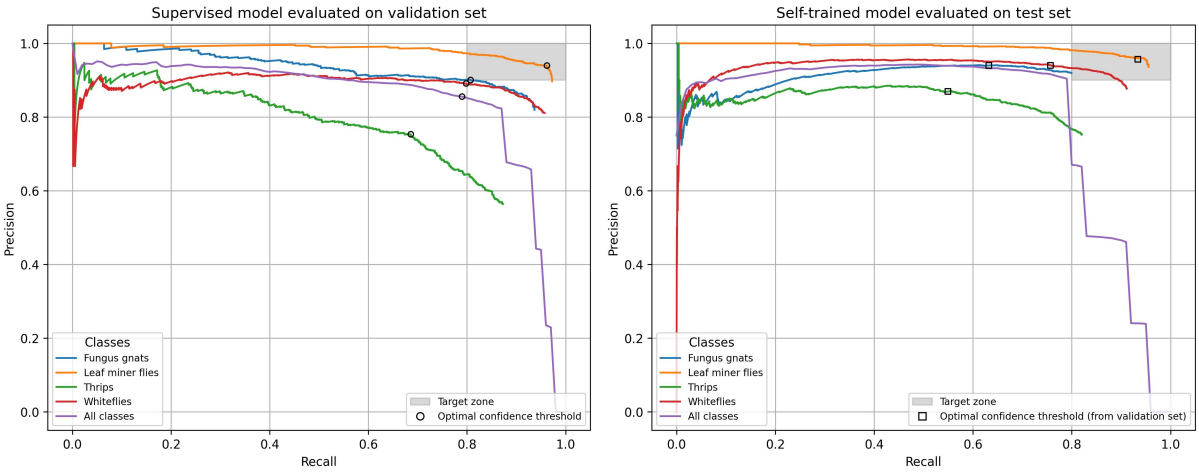


Figure 12: Precision-recall curves for supervised training on tiles. The results are slightly worse than supervised training on full images (see 8) for all but the thrips. Again the model suffers from the confidence thresholds derived from the insufficiently labeled validation set.

3.4 Image processing

The first subsection gives the results for testing the different masks that are needed to filter out background structures. In the second subsection the results for the different sets of thresholds that are needed to filter the rectangles created by the image processing are reported.

3.4.1 Preprocessing for segmentation

Topic sentence: The masks are used to filter out the background structures from the image before segmenting it to find the insects. The best mask decreased the number of false positives while retaining a sufficient number of true positives for the self-training.

Figures&Tables: Results for the different masks and the application procedures can be compared in Figure 13. Thresholds used for creating the results are found in the methods section, Table 6, filter set 1. Using this filter set and the mask with the best results, statistics where taken and used as threshold for filter set 2 (see Table 11).

Exact results in (mask_testing_collected_results.csv)

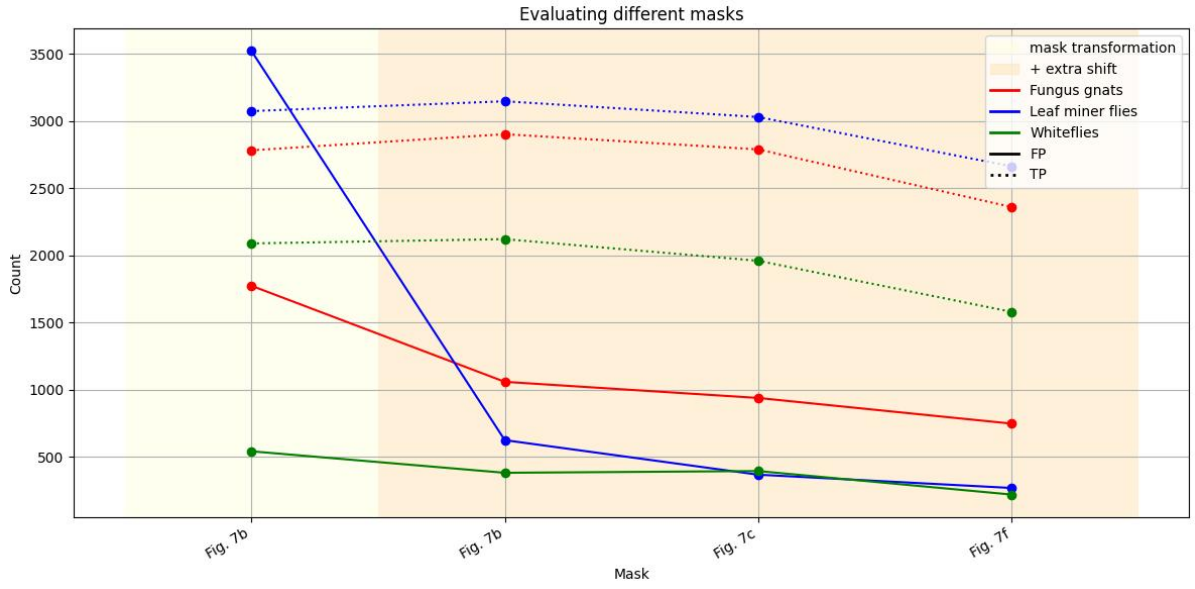


Figure 13: Evaluating the masks given in Figure 7. The thicker lines of mask 8c in combination with the transformation (i.e. adapting the mask to each single image) reduces false positives by a large margin. Even thicker lines tend to also decrease the true positives. The extra shift of the mask mainly reduces the number of false positives for the leaf miner flies.

Training Set	Metric	Fungus Gnats	Leaf Miner Flies	Whiteflies
Labeled Training Set	95th percentile box size	42260	23544	3275
	95th percentile box ratio	1.74	1.77	1.88
Unlabeled Training Set	95th percentile box size	43456	28480	3477
	95th percentile box ratio	1.73	1.76	1.84

Table 11: Found possible thresholds for labeled and unlabeled training set.

The mask with the best results (Figure 7d) produced distributions of boxes sizes, deviations from 1:1 ratio of sidelenghts and distribution of values for the whitefly rectangles, all taken from the results of the simple filter set 1 (see Table 6. Table 12 gives the found values for the metrics for both the labeled and the unlabeled training set, which are similar as expected.

3.4.2 Segmentation

Applying a refined filter set with thresholds derived by statistics reduced the number of false positives further and increased precision while retaining a sufficient number of true positives for the following self-training.

Figure 14 shows the results of applying the filters. Table 12 shows the resulting numbers of rectangles created by the image processing.

Figure 14 shows the results for the tests with the found thresholds. The two filter sets 1 and 2 as given in Table 6 are always the first and last steps in the graphs. In between single changes to set 1 are visualized. For the fungus gnats, all tried changes yield smaller effects by tolerable reducements of the number of true positives and were combined in the end. The number of false positives is still the highest compared to the other classes, which as already observed in the evaluation of the supervised training might be mainly due to missing labels. For the leaf miner flies, the effect are comparable but less prominent as in the fungus gnats, but precision and number of false positives were also already acceptable only with the first filter set. For the whiteflies, filtering the rectangles by box size and ratio was not taken over into the filter set, because it reduced the number of already scarce true positives too much. Instead overlapping boxes were deleted and filtering by value of the rectangles reduced false positives and at the same time retained the number of true positives.

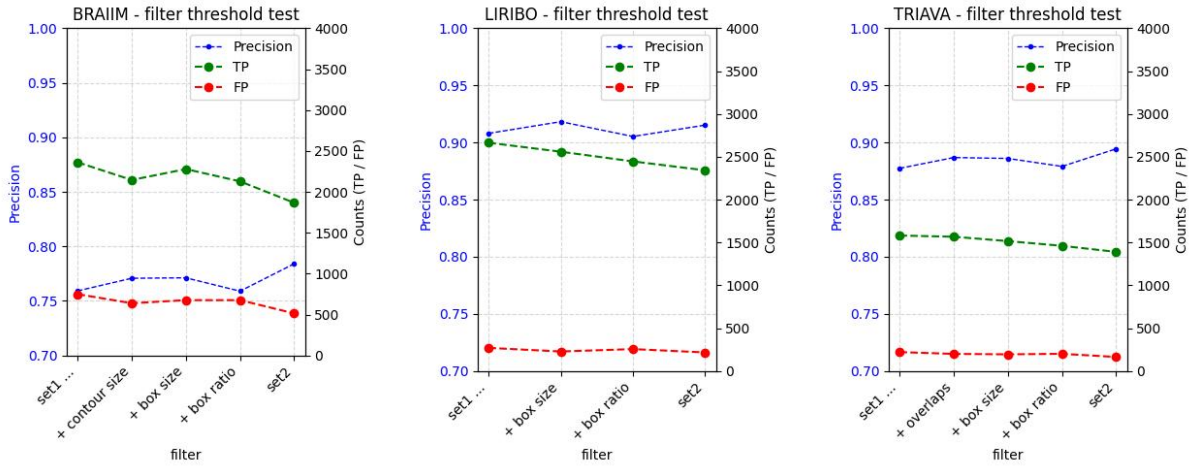


Figure 14: Results for different filters for each species. Leftmost is always the parameter set 1 with no extra filtering and estimated thresholds, rightmost is parameter set 2 with derived threshold and all extra filters applied. In between single additions to set 1 are tested. Species do not follow my usual naming convention

As to be expected objects of the same color, shape and size cannot be filtered out. This concerns mainly parts of the insects as well as malformed or very untypical instances, as well as insects occluding each other. Table 12 gives the absolute numbers of rectangles returned which each filter set for the labeled and unlabeled training set. As expected they turn out to be very similar with exception of the total numbers of whiteflies, which is doubled in the unlabeled training set. As the number of filtered out rectangles is also proportionally larger, this is possibly only a effect of more crowded whitefly images in the unlabeled set.

Class	Labeled training set		Unlabeled training set	
	Filter set 1	Filter set 2	Filter set 1	Filter set 2
Fungus gnats	3110	2380	3477	2699
Leaf miner flies	2934	2684	3116	2628
Whiteflies	1802	1575	5015	4308

Table 12: Absolute number of rectangles created.

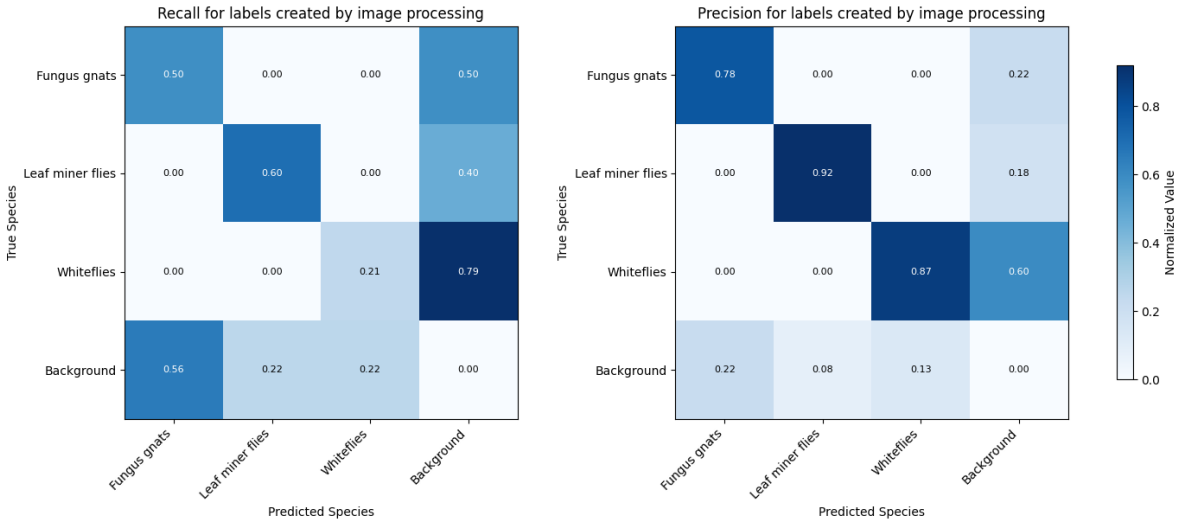


Figure 15: Confidence matrices for the image processed masks. Low recall is mainly due to the insects on the background structures that were masked out.

3.5 Self training

A model trained on the augmented data sets 1 & 2 as full images learned the class distinctions and also predicts more individuals, especially for objects on background structures (compare Fig 11 in the final results with the Fig 15 above). The model is used to predict pseudo labels and is then trained on the original images, which improves its performance only slightly and only for one epoch.

- The best model trained on the labels created by image processing used the following parameters: 20 epochs, parameterset 2 with additional flipud augmentation set top 0.5, and both augmentation set 1 and 2 combined - same setting for full images and tiles (which included some emtpy tiles)
- The best model was used to predict on the original images. Predictions were used as pseudo labels with confidence thresholds taken from visual inspection of the predictions. Then the model was trained for another 20 epochs, but now on the original data. This was repeated two times (for full images) and three times (for tiles, train12).

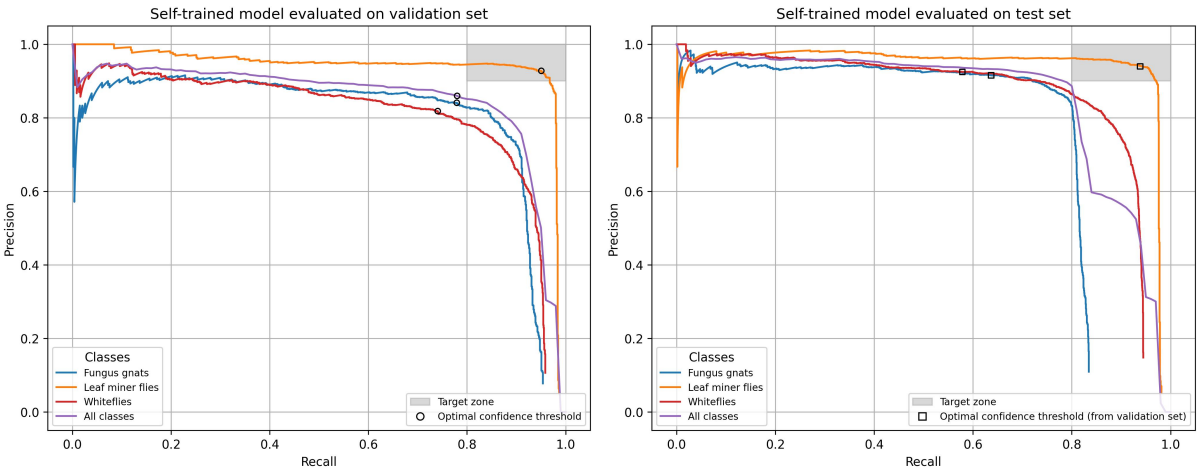


Figure 16: Precision-recall curves for the best model self-trained on tiles. The left curve is taken on the validation set with the operating points closest to the target zone. The right curve is taken on the test set with the operating points taken from the validation set. The results are worse then training on full images (compare Fig 10)

4 Discussion

- relate labeling quality to results (see table with improved dataset)
- for single application, the brute force method is the most efficient solution given coherent and complete labeling for at least the validation and test set.
- for tables without background and with clean objects and good contrast, the image processing/self training approach will possibly work much better
- Quality of the data is key: no background, clean images, correct labeling
- Other problems still to be solved: Final images have lower res (domain shift), individuals sinking into the glue and changing their appearance
- grounding models might be a viable alternative
- not yet tried: tile size 1280. doing fully automated self training.
- creating pseudo-labels from the labels created by image processing works seamless - after one training one them the models detects can distinct between classes and detects also the insects on background structures. However confidence thresholds do not allow to distinguish between true and false positives, e.g. foliage is detected as insect with high confidence. This prevents the model to predict good pseudo-labels. As the leaf miner set shows, clean images are key for good results.

5 Conclusion

6 References

[1] Curtis A. Deutsch et al. "Increase in crop losses to insect pests in a warming climate". In: *Science* 361.6405 (2018), pp. 916–919. DOI: 10.1126/science.aat3466.

[2] Laure Mamy et al. *Impacts des produits phytopharmaceutiques sur la biodiversité et les services écosystémiques. Rapport de l'expertise scientifique collective*. Research Report. INRAE ; IFREMER, 2022, 1408 p. DOI: 10.17180/0gp2-cd65.

[3] Matthew R. Smith et al. "Pollinator Deficits, Food Consumption, and Consequences for Human Health: A Modeling Study". In: *Environmental Health Perspectives* 130.12 (2022), p. 127003. DOI: 10.1289/EHP10947.

[4] URL: https://www.fao.org/pest-and-pesticide-management/ipm/integrated-pest-management/en/ (visited on 04/04/2025).

[5] G.J. Messelink and H.M. Kruidhof. "Advances in pest and disease management in greenhouse cultivation". In: *Achieving sustainable greenhouse cultivation*. Ed. by L. Marcelis and E. Heuvelink. United Kingdom: Burleigh Dodds Science Publishing, Sept. 2019, pp. 311–356. DOI: 10.19103/as.2019.0052.17.

[6] URL: https://food.ec.europa.eu/plants/pesticides/sustainable-use-pesticides/integrated-pest-management-ipm_en (visited on 04/02/2025).

[7] Tiago Domingues, Tomás Brandão, and João C. Ferreira. "Machine Learning for Detection and Prediction of Crop Diseases and Pests: A Comprehensive Survey". In: *Agriculture* 12.9 (2022). DOI: 10.3390/agriculture12091350.

[8] Ana Cláudia Teixeira et al. "A Systematic Review on Automatic Insect Detection Using Deep Learning". In: *Agriculture* 13.3 (2023). DOI: 10.3390/agriculture13030713.

[9] Mamta Mittal et al. "Machine learning for pest detection and infestation prediction: A comprehensive review". In: *WIREs Data Mining and Knowledge Discovery* 14.5 (2024). DOI: https://doi.org/10.1002/widm.1551.

[10] Joseph Redmon et al. "You only look once: Unified, real-time object detection". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 779–788. DOI: 10.48550/arXiv.1506.02640.

[11] Dan Jeric Arcega Rustia et al. "Automatic greenhouse insect pest detection and recognition based on a cascaded deep learning classification method". In: *Journal of Applied Entomology* 145.3 (Apr. 2021), pp. 206–222. DOI: 10.1111/jen.12834.

[12] Wenyong Li et al. “Field detection of tiny pests from sticky trap images using deep learning in agricultural greenhouse”. In: *Computers and Electronics in Agriculture* 183 (Apr. 2021), p. 106048. DOI: 10.1016/j.compag.2021.106048.

[13] Dinis Costa et al. “Intelligent System for Automatic Whitefly Detection in Tomato Greenhouses”. In: *2023 6th Experiment@ International Conference (exp.at'23)*. Évora, Portugal: IEEE, June 2023, pp. 29–30. DOI: 10.1109/exp.at2358782.2023.10546195.

[14] Xiaolei Zhang et al. “Automatic pest identification system in the greenhouse based on deep learning and machine vision”. In: *Frontiers in Plant Science* 14 (Sept. 2023), p. 1255719. DOI: 10.3389/fpls.2023.1255719.

[15] Song Wang et al. “A Deep-Learning-Based Detection Method for Small Target Tomato Pests in Insect Traps”. In: *Agronomy* 14.12 (Dec. 2024). Publisher: MDPI AG, p. 2887. DOI: 10.3390/agronomy14122887.

[16] Jianyu Shi et al. “Small Target Insect Detection Based on Improved YOLOv8n”. In: *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Hyderabad, India: IEEE, Apr. 2025, pp. 1–5. DOI: 10.1109/icassp49660.2025.10890801.

[17] Dan Jeric Arcega Rustia et al. “Online semi-supervised learning applied to an automated insect pest monitoring system”. In: *Biosystems Engineering* 208 (Aug. 2021), pp. 28–44. DOI: 10.1016/j.biosystemseng.2021.05.006.

[18] Xianwei Li et al. “MATeacher: Multi-scale Views Learning and Adaptive Unsupervised Weight for Semi-supervised Pest Region Detection”. In: *2024 6th International Conference on Robotics, Intelligent Control and Artificial Intelligence (RICAI)*. Nanjing, China: IEEE, Dec. 2024, pp. 698–703. DOI: 10.1109/RICAI64321.2024.10911445.

[19] Paweł Majewski et al. “Improved Pest Detection in Insect Larvae Rearing with Pseudo-Labelling and Spatio-Temporal Masking:” in: *Proceedings of the 19th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. Rome, Italy: SCITEPRESS - Science and Technology Publications, 2024, pp. 349–356. DOI: 10.5220/0012311300003660.

[20] Jiale Zhou et al. “Mutual learning with memory for semi-supervised pest detection”. In: *Frontiers in Plant Science* 15 (June 2024), p. 1369696. DOI: 10.3389/fpls.2024.1369696.

[21] Cheng Zhou and Fanhuai Shi. “Perceptive Teacher: Semi-Supervised small object detection of Brassica Chinensis seedlings and pest infestations”. In: *Computers and Electronics in Agriculture* 229 (Feb. 2025), p. 109956. DOI: 10.1016/j.compag.2025.109956.

[22] Laura Gomez-Zamanillo et al. “Semi-Supervised Approach for Automatic Counting of Whiteflies With Small Annotated Dataset”. In: *IEEE Access* 13 (2025), pp. 55586–55598. DOI: 10.1109/ACCESS.2025.3552195.

[23] L. Minh Dang et al. “An efficient zero-labeling segmentation approach for pest monitoring on smartphone-based images”. In: *European Journal of Agronomy* 160 (Oct. 2024), p. 127331. DOI: 10.1016/j.eja.2024.127331.

[24] Waldemar Raaz et al. “Smart Checkpots–Proof of concept for a mobile pest and climate monitoring system in greenhouses”. In: *DGG-Proceedings* 11.8 (2023). DOI: 10.5288/DGG-PR-11-08-WR-2023.

[25] Alexander E. Dearden et al. “Visual modelling can optimise the appearance and capture efficiency of sticky traps used to manage insect pests”. In: *Journal of Pest Science* 97 (2024), pp. 469–479. DOI: https://doi.org/10.1007/s10340-023-01604-w.